

Import Libraries

```
In [1]: import seaborn as sns
import pandas as pd
import numpy as np
```

Load Dataset

```
In [2]: df = sns.load_dataset('iris')
```

```
In [3]: print(df.head())
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
In [4]: print(df.shape)
```

(150, 5)

```
In [5]: print(df.isnull().sum())
```

```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

```
In [6]: print(df['species'].unique())
```

['setosa' 'versicolor' 'virginica']

Convert to Binary

```
In [7]: df = df[df['species'] != 'setosa']
```

```
In [8]: print(df.shape)
```

(100, 5)

Create Dependent & Independent Variables

```
In [9]: X = df.iloc[:, :-1]
```

```
In [10]: y = df.iloc[:, -1]
```

Train-Test Split

```
In [11]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

```
In [12]: print(X_train.shape, X_test.shape)
```

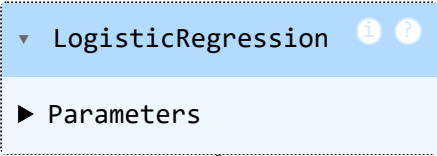
```
(75, 4) (25, 4)
```

Model Building

```
In [13]: from sklearn.linear_model import LogisticRegression
```

```
In [14]: model = LogisticRegression()
```

```
In [15]: model.fit(X_train, y_train)
```

```
Out[15]: 
```

Prediction

```
In [16]: y_pred = model.predict(X_test)
```

```
In [17]: print("Predictions:", y_pred)
```

```
Predictions: ['virginica' 'virginica' 'virginica' 'versicolor' 'versicolor'
'versicolor' 'versicolor' 'virginica' 'versicolor' 'versicolor'
'versicolor' 'versicolor' 'virginica' 'virginica' 'virginica'
'versicolor' 'virginica' 'virginica' 'versicolor' 'versicolor'
'virginica' 'versicolor' 'versicolor' 'versicolor' 'virginica']
```

Evaluation

```
In [18]: from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, r
```

Task 1: Predict Class using User Input and Graph Plot

```
In [19]: class IrisPredictor:
    def __init__(self, model):
        self.model = model

    def predict_species(self, sl, sw, pl, pw):
        #use dataframes(with same column names)
        data = pd.DataFrame([[sl, sw, pl, pw]],
                             columns=['sepal_length', 'sepal_width', 'petal_lengt

        pred = self.model.predict(data)

        if pred[0] == 0:
            return "versicolor"
        else:
            return "virginica"

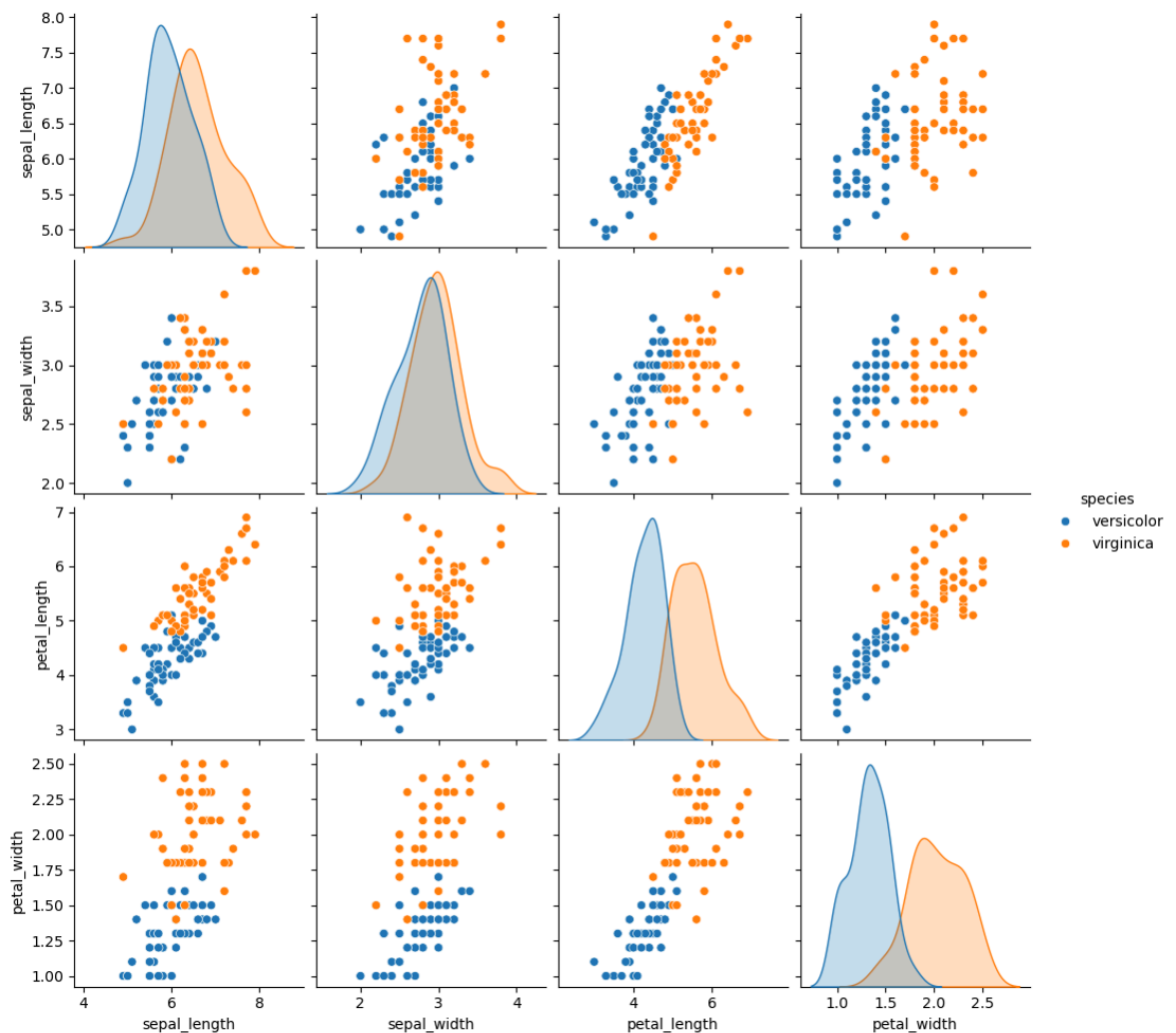
predictor = IrisPredictor(model)

result = predictor.predict_species(6.0, 2.9, 4.5, 1.5)
print("Predicted Species:", result)
```

Predicted Species: virginica

```
In [20]: import matplotlib.pyplot as plt
```

```
In [21]: sns.pairplot(df, hue='species')  
plt.show()
```



Task 2: Apply Same Steps on Another Dataset

```
In [54]: import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LogisticRegression  
from sklearn.preprocessing import LabelEncoder
```

Load dataset

```
In [55]: df = pd.read_csv("Breast_Cancer.csv")
```

```
In [56]: print(df.head())
```

	Age	Race	Marital Status	T Stage	N Stage	6th Stage	\
0	68	White	Married	T1	N1	IIA	
1	50	White	Married	T2	N2	IIIA	
2	58	White	Divorced	T3	N3	IIIC	
3	58	White	Married	T1	N1	IIA	
4	47	White	Married	T2	N1	IIB	

	differentiate	Grade	A Stage	Tumor Size	Estrogen Status	\
0	Poorly differentiated	3	Regional	4	Positive	
1	Moderately differentiated	2	Regional	35	Positive	
2	Moderately differentiated	2	Regional	63	Positive	
3	Poorly differentiated	3	Regional	18	Positive	
4	Poorly differentiated	3	Regional	41	Positive	

	Progesterone Status	Regional Node Examined	Reginol Node Positive	\
0	Positive	24	1	
1	Positive	14	5	
2	Positive	14	7	
3	Positive	2	1	
4	Positive	3	1	

	Survival Months	Status
0	60	Alive
1	62	Alive
2	75	Alive
3	84	Alive
4	50	Alive

```
In [57]: le = LabelEncoder()
```

```
In [58]: for col in df.columns:
          if df[col].dtype == 'object':
              df[col] = le.fit_transform(df[col])
```

```
In [59]: # Features and target
X = df.drop('Status', axis=1)
y = df['Status']
```

```
In [60]: # Train test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
In [61]: # Train model
model = LogisticRegression(max_iter=10000)

model.fit(X_train, y_train)

print("Model Trained Successfully")
```

Model Trained Successfully

```
In [62]: class CancerPredictor:
          def __init__(self, model):
              self.model = model

          def predict_status(self, features):

              data = pd.DataFrame([features], columns=X.columns)
```

```

pred = self.model.predict(data)

if pred[0] == 0:
    return "Alive"
else:
    return "Dead"

predictor = CancerPredictor(model)

# Example row from dataset
sample = X.iloc[0].tolist()

result = predictor.predict_status(sample)

print("Prediction:", result)

```

Prediction: Alive

In [63]: `import matplotlib.pyplot as plt`

In [64]: `sns.pairplot(df, hue='Status')`
`plt.show()`

